

Programozás Alapjai

Dr. Gergely Tamás
Dr. Kicsi András

Szegedi Tudományegyetem
Informatikai Intézet
Szoftverfejlesztés Tanszék

2025

(v1207)



Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Péneváltás (1. verzió)
 - Péneváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listáshoz tartozó műveletek
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszóátlag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtovábbai pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - 23 Absztrakt adattípusok**
 - További Python típusok
 - Python function adattípus
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Példák (1. verzió)
 - Példák (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programozási fogalmak
 - 4 Python feltételes ciklusok**
 - Röppályák számítás
 - Röppályák szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python listák és tuple adat típusok
 - Python listák szűrése
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszólagos számítás
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adat típusok
 - Tér legtovábbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adat típus
 - Python comprehension
 - Python function adat típus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - 23 További Python típusok**
 - További Python típusok
 - Python function adat típus
 - 23 Absztrakt adattípusok**
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

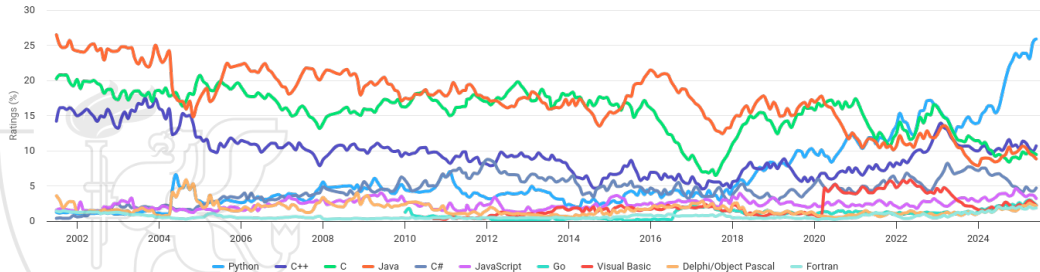
- A félév elején az előadások első felében inkább elméleti oldalról közelítjük a szoftverfejlesztést és a Python programozást.
 - Példákon keresztül megnézzük, hogyan lehet problémákat modellezni, megoldani, a megoldást lekódolni (Python nyelven), áttekintjük az elméletet.
- A félév elején az előadások második felében gyakorlatias példákon keresztül áttekintjük a Python nyelvet – egy kis részét, amiről ezen a kurzuson szó lesz –, kifejezetten a nyelvi megvalósításra koncentrálva, bővebb elméleti magyarázat nélkül.
 - A cél az, hogy a következő hét gyakorlatain már alkalmazni lehessen az áttekintett nyelvi elemeket.
- A félév vége felé speciálisabb témák és mélyebb magyarázatok kerülnek elő.
 - Mondanám, hogy leásunk a Python nyelv mélyére, de nem tesszük; most csak a felszínt fogjuk kapargatni.

Miért a Python programozási nyelv?

- 2025 júniusában 25.9%-kal a Python volt a legnépszerűbb programozási nyelv (a 10.7%-os C++, 9.5%-os C és 8.8%-os Java előtt).

TIOBE Programming Community Index

Source: www.tiobe.com



(2025 június) <https://www.tiobe.com/tiobe-index/>

A Python nyelv

Történelem [3]

- Guido van Rossum fejlesztette ki az 1980-as évek vége felé (az ABC és SETL nyelvekből merített).
- 1989 decemberében kezdődött a tényleges implementáció.
- A **Python 1.0** 1994. január 26-án jelent meg.
- A **Python 2.0** 2000. október 16-án jelent meg, jelentősen bővítve a nyelvet és a környezetet.
- A **Python 3.0** 2008. december 3-án jelent meg, a nyelvi fejlesztések miatt nem kompatibilis a korábbi verziókkal.
- 2018 óta egyre nagyobb népszerűségnek örvend.
- A legutóbbi verziók nagyon sokat javítottak a Python hatékonyságán (sebességén).
 - De még mindig könnyű borzalmasan lassú programokat írni benne.

Miért népszerű?

„Zen of Python” [4]

- Beautiful is better than ugly.
- Explicit is better than implicit.
- Simple is better than complex.
- Complex is better than complicated.
- Flat is better than nested.
- Sparse is better than dense.
- Readability counts.
- Special cases aren't special enough to break the rules.
- Although practicality beats purity.
- Errors should never pass silently.
- Unless explicitly silenced.
- In the face of ambiguity, refuse the temptation to guess.
- There should be one – and preferably only one – obvious way to do it.
- Although that way may not be obvious at first unless you're Dutch.
- Now is better than never.
- Although never is often better than **right** now.
- If the implementation is hard to explain, it's a bad idea.
- If the implementation is easy to explain, it may be a good idea.
- Namespaces are one honking great idea – let's do more of those!

A Python nyelv

Tulajdonságok

- „Emberközeli” nyelv: a kifejezései hasonlítanak a természetes nyelvi (és a matematikai) kifejezésekhez.
- Dinamikus szkriptnyelv, de fordított.
- Funkcionális programozási elemek (pl. *list*, *lambda*, *map*, *reduce*, ...).
- Sok beépített függvény.
- Rengeteg modul, és egy komplex csomagkezelő környezet is jár hozzá.
- Nem típusolt változók, az értékeknek (objektumoknak) vannak típusai.
- Összetettebb típusok: *complex*, *list*, *set*, *dictionary*, ...
- Objektorientáltság, kivételkezelés, felülírható mechanizmusok.
- C/C++ API
 - Hatékony (gyors) C/C++ modulok Python programokhoz való integrálására (pl. MI).
 - Python szkriptek futtatására C/C++ programokban (pl. játékok).

- Referenciák:

- <https://docs.python.org/3/reference/index.html>
- https://www.w3schools.com/python/python_reference.asp
- ...

- Tutorial-ok:

- <https://docs.python.org/3/tutorial/index.html>
- <https://www.w3schools.com/python/>
- <https://www.tutorialspoint.com/python/index.htm>
- <https://www.pythontutorial.net/>
- <https://www.geeksforgeeks.org/python-programming-language-tutorial/>
- <https://python-course.eu/>
- ...

Python fejlesztői környezetek

- Integrated Development Environments
 - PyCharm
 - VS code
 - IDLE (a Python saját környezete)
- „Fapados” környezetek
 - Text alapú szövegszerkesztő (vi, mcedit, gedit, kate, geany, subl)
 - + parancssori futtatás:

```
$ python3 program.py
```

- A python interpreter (interaktívan használva)

```
$ python3
>>> print("Hello!")
Hello!
>>> exit()
$
```

Hello Világ!

hello_min.py [1-1]

```
1 print("Helló_Világ!")
```



Hello Világ!

hello.py [1–10]

```
1#!/usr/bin/env python3
2"""
3Programozás, Alapjai, előadás, példaprogram
4SZTE, TTIK, Informatikai Intézet, Szoftverfejlesztés, Tanszék
5Gergely, Tamás, gertom@inf.u-szeged.hu, 2024-05-31
6cc-by-nc-nd
7
8'Hello, World!' program, (majdnem) minimális megvalósítás
9"""
10print("Helló, Világ!")
```



Hello Világ!

hello_main.py [1–14]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-05-31
6cc-by-nc-nd
7
8'Hello World!' program, main függvényben
9"""
10def main():
11    print("Helló Világ!")
12
13if __name__ == "__main__":
14    main()
```

- A `hello_main.py` program végrehajtása parancssorból:

```
$ python3 hello_main.py
Helló Világ!
$
```

- Ha a Python szkriptünk

- első sora fix formátumú ('#!'-lel, úgynevezett *shebang*-gel kezdődő) komment sor, és
- ebben a megfelelő Python interpreter van hivatkozva, és
- van a programfájlon futtatási jog,

akkor azt (valamilyen linux alapú rendszeren) egyszerű futtatható programként is indíthatjuk, például:

```
$ ./hello.py
Helló Világ!
$
```

Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)**
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Algoritmus vezérlése
 - A Python nyelv utasításai
 - 7 Vezérlési szerkezetek**
 - Python sztringek
 - Basename meghatározása
 - 8 Python listák**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 9 Vezérlési szerkezetek**
 - Python list és tuple adattípusok
 - Python listáshoz adattípusok műveletei
 - Basename és eltelt idő
 - 10 Python iterációk**
 - Csúszóátlag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtovábbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - További Python típusok
 - Python function adattípus
 - 23 Absztrakt adattípusok**
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

Pénzváltás

penzvalto.py (v1.0) [1–16]

```
1#!/usr/bin/env python3
2"""
3Programozás, Alapjai, előadás, példaprogram
4SZTE, TTIK, Informatikai Intézet, Szoftverfejlesztés, Tanszék
5Gergely, Tamás, gertom@inf.u-szeged.hu, 2024-05-31
6cc-by-nc-nd
7
8Pénzváltás, rögzített, árfolyamon
9"""
10def main():
11    forint = float(input("Hány forintot váltatsunk? "))
12    euro = forint / 396.717 # 2025-08-28
13    print(f"{forint:.0f} HUF = {euro:.2f} EUR")
14
15if __name__ == "__main__":
16    main()
```

Pénzváltás

A probléma

- Van valamennyi pénzünk forintban.
- Mennyi Eurót tudunk vásárolni érte?



Egyszerű kivitel

`print()`

- `print(...)`:
 - Python „beépített” (*built-in*) függvény
 - Alapvetően a standard kimenetet (`sys.stdout`) használja, ez alapesetben a terminál / képernyő
 - **0**: Kiírja az argumentumként kapott sztringet
 - **1**: Kiírja az argumentum értékét, sztringgé alakítva
 - **2**: Kiírja az argumentumok értékét egy (sorvége jellel lezárt) sorban, szóközzel elválasztva

```
>>> print()

>>> print('Hello!')
Hello!

>>> print(2024)
2024

>>> y=2024
>>> h="Hi!"
>>> y
2024

>>> print(h, y)
Hi! 2024
```

Egyszerű bevitel

`input()`

- `input(...)`:
 - Python „beépített” (*built-in*) függvény
 - Alapvetően a standard bemenetet (`sys.stdin`) használja, ez alapesetben a billentyűzet
 - **0**: Visszaadja a beolvasott teljes sort (sorvége jel nélkül) sztringként
 - **1**: Megadhatunk egy sztringet, amit a beolvasás előtt kiír („prompt”)

```
>>> input()
hello
'hello'
>>> input('Kérdés?_')
Kérdés? van
'van'
>>> n = input('n_értéke:_')
n értéke: 6
>>> print(n)
6
>>> n
'6'
```

Sztring literálok

- A sztringek (szövegek, `str`) Python-ban egyfajta listák, karakterek listái
 - *Ez így nem teljesen igaz, pythonban nincs karakter típusú érték, csak 1 hosszúságú sztring!*
- Sztring literálokat (értékeket) többféleképpen lehet megadni
 - Aposztrófok (`'` – *quote*) között
 - Idézőjelek (`"` – *double quote*) között
 - Többsoros sztringeket tripla határolók (`'''` vagy `"""` – „*triple quote*”) között

```
>>> 'Ez_egy_aposztrófok_között_megadott_sztring.'  
'Ez_egy_aposztrófok_között_megadott_sztring.'  
>>> "Ez_egy_idézőjelek_között_megadott_sztring."  
'Ez_egy_idézőjelek_között_megadott_sztring.'  
>>> """Ez_pedig,  
..._mint_az_látszik,  
..._egy_többsorosként_megadott_sztring."""  
'Ez_pedig,\n mint_az_látszik,\negy_többsorosként_megadott_sztring.'
```

Egy Python program felépítése

Interaktív használat

- Amikor interaktív módon használjuk a Python interpretert, akkor
 - egész egyszerűen egymás után adjuk ki az utasításokat,
 - amiket az interpreter azonnal végrehajt, és
 - megjeleníti az eredményt (ha van).

```
>>> print('Helló!') # Van kimenet, nincs eredmény (az eredmény: None)
Helló!
>>> 6 * 7          # Nincs direkt kimenet, van eredmény
42
>>> answer = 6 * 7 # Nincs se direkt kimenet, se eredmény
>>> str(answer)    # Nincs eredmény, van direkt kimenet
'42'
```

Egy Python program felépítése

Egyszerűbb program, nem interaktív használat

- Amikor **nem** interaktív módon használjuk a Python interpretert, akkor
 - egész egyszerűen egymás után írjuk az utasításokat,
 - amiket az interpreter sorban végrehajt.
 - Az egyes utasítások *eredményei* nem jelennek meg automatikusan.
- Egy egyszerű Python program így néz ki:

```
1#!/usr/bin/env python3
2"""
3Ez itt egy úgynevezett 'docstring', technikailag egy sztring literál.
4Ide kerülhetnek a program meta-adatai (ki készítette, mikor, mit csinál a program, licensz, stb.),
5dokumentációja.
6A program futását nem befolyásolja, de az interpretert interaktívan --és a programunkat modulként--
7használva hasznos lehet a 'help()' függvényhez. Ilyen 'docstring'-et egyébként majd egyes programelemekhez
8is tudunk készíteni.
9"""
10
11Utasítás1
12Utasítás2
13Utasítás3
14...
```

- Olvassunk be egy Forint összeget!
- Tároljuk el!
- Ellenőrizzük le, hogy mit kaptunk!



Pénzváltás

penzvalto.py (v0.1) [1–12]

```
1#!/usr/bin/env python3
2"""
3Programozás, Alapjai, előadás, példaprogram
4SZTE, TTIK, Informatikai Intézet, Szoftverfejlesztés, Tanszék
5Gergely, Tamás, gertom@inf.u-szeged.hu, 2024-07-01
6cc-by-nc-nd
7
8Pénzváltás, rögzített, árfolyamon
9"""
10forint = input("Hány forintot váltssunk? ")
11# TESZT
12print(type(forint), ':', forint)
```



A Python nyelv beépített adattípusai

Szöveg	str	Egy karaktersorozat ábrázolására való.
Szám	int, float, complex	Egész, valós és komplex számok ábrázolására való típusok.
Sorozat	list, tuple, range	Értékek sorozatának ábrázolására való típusok.
Leképezés	dict	Kulcs-érték párok ábrázolására való típus.
Halmaz	set, frozenset	Értékek halmazának ábrázolására való típusok.
Logikai	bool	Logikai értékek (True és False) ábrázolására való típus.
Bináris	bytes, bytearray, memoryview	Bináris adatok ábrázolására való típusok.
Semmi	NoneType	A None érték, azaz a semmi típusa.
		Nincs önálló karakter (char) és bájt (byte) típus!

Típuskonverziók Pythonban

- Gyakran szükséges, hogy valamilyen típusú értéket más típusúra konvertáljunk
- Automatikus (*implicit*) konverziók, pl: `int`→`float`, `list`→`bool`, `int`→`bool`, ...

```
>>> 1234567890123456789 + 1.0
1.2345678901234568e+29
>>> 'nemüres' if [1,2,3] else 'üres'
'nemüres'
>>> 'nemnulla' if 0 else 'nulla'
'nulla'
```

- Kényszerített (*explicit*) konverziók, pl: `float`→`int`, `str`→`int`, `float`→`str`, ...

```
>>> int(1234567890123456789 + 1.0)
12345678901234567168
>>> int('42')
42
>>> str(0.3e3)
'300.0'
```

- Olvassunk be egy Forint összeget!
- Alakítsuk át (valós) szám típusúvá!
- Tároljuk el!
- Ellenőrizzük le, hogy mit kaptunk!



Pénzváltás

penzvalto.py (v0.2) [1–12]

```
1#!/usr/bin/env python3
2"""
3Programozás, Alapjai, előadás, példaprogram
4SZTE, TTIK, Informatikai Intézet, Szoftverfejlesztés, Tanszék
5Gergely, Tamás, gertom@inf.u-szeged.hu, 2024-07-01
6cc-by-nc-nd
7
8Pénzváltás, rögzített, árfolyamon
9"""
10forint = float(input("Hány forintot váltstunk? "))
11# TESZT
12print(type(forint), ':', forint)
```



Műveletek számokkal

Matematikai kifejezések

- Az `int` adattípus műveleteire teljesülnek az aritmetika ismert azonosságai.

```
>>> 2 * (3 * 7 + 5 * 2 ** 2) == 2 ** 2 * 2 * 2 * 5 + 3 * 2 * 7
True
```

- A `float` adattípus műveleteire csak adott értéktartományban és adott pontossággal teljesülnek az aritmetika ismert azonosságai⁶.

```
>>> math.sqrt(2.0) ** 2
2.0000000000000004
>>> 3.1415927e20 + 2.718281e-20 == 3.1415927e20
True
>>> 1.7976931348623157e+308 + 1e292
inf
```

⁶A Python az IEEE-754 szabvány [5] „double precision” típusát valósítja meg

- Olvassunk be egy Forint összeget!
- Alakítsuk át (valós) szám típusúvá!
- Tároljuk el!
- Számoljuk ki, hány Euro-t ér, valamilyen rögzített Euroárfolyamon!
- Tároljuk el ezt az értéket is!
- Írassuk ki az Euro értéket!



Pénzváltás

penzvalto.py (v0.3) [1–12]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-07-01
6cc-by-nc-nd
7
8Pénzváltás rögzített árfolyamon
9"""
10forint = float(input("Hány forintot váltstunk? "))
11euro = forint / 396.717 # 2025-08-28
12print(euro)
```



- Olvassunk be egy Forint összeget!
- Alakítsuk át (valós) szám típusúvá!
- Tároljuk el!
- Számoljuk ki, hány Euro-t ér, valamilyen rögzített Euroárfolyamon!
- Tároljuk el ezt az értéket is!
- Írassuk ki hogy a megadott Forint összeg hány Euro-t ér szép, „x HUF = y EUR” formában!

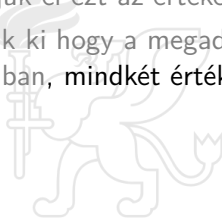
Pénzváltás

penzvalto.py (v0.4) [1–12]

```
1#!/usr/bin/env python3
2"""
3Programozás, Alapjai, előadás, példaprogram
4SZTE, TTIK, Informatikai Intézet, Szoftverfejlesztés, Tanszék
5Gergely, Tamás, gertom@inf.u-szeged.hu, 2024-07-01
6cc-by-nc-nd
7
8Pénzváltás, rögzített, árfolyamon
9"""
10forint = float(input("Hány forintot válttsunk? "))
11euro = forint / 396.717 # 2025-08-28
12print(forint, "HUF", euro, "EUR")
```



- Olvassunk be egy Forint összeget!
- Alakítsuk át (valós) szám típusúvá!
- Tároljuk el!
- Számoljuk ki, hány Euro-t ér, valamilyen rögzített Euroárfolyamon!
- Tároljuk el ezt az értéket is!
- Írassuk ki hogy a megadott Forint összeg hány Euro-t ér szép, „x HUF = y EUR” formában, mindkét értéket megfelelő pontossággal!



Pénzváltás

penzvalto.py (v0.5) [1–12]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTIK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-07-01
6cc-by-nc-nd
7
8Pénzváltás rögzített árfolyamon
9"""
10forint = float(input("Hány forintot váltstunk? "))
11euro = forint / 396.717 # 2025-08-28
12print(f"{forint:.0f} HUF = {euro:.2f} EUR")
```



Egy Python program felépítése

Good practice

- Mivel egy Python programot – más Python programokból – modulként is lehet használni, nem mindig jó ötlet, hogy a „főprogramunk” olyankor is lefut.
- Ennek elkerülésére a következő minta használható:

```
1#!/usr/bin/env python3
2"""
3Python 'goodpractice' példa.
4"""
5def main():
6    """
7    Ez a függvényünk veszi át a főprogram szerepét, és a lenti feltétel miatt csak akkor fut le, ha a
8    modult (Python fájl) közvetben, programként hívjuk meg; ha modulként, 'import'-on keresztül
9    használjuk, akkor nem.
10    """
11    Utasítás1
12    Utasítás2
13    Utasítás3
14    ...
15
16if __name__ == '__main__':
17    main()
```

- Olvassunk be egy Forint összeget!
- Alakítsuk át (valós) szám típusúvá!
- Tároljuk el!
- Számoljuk ki, hány Euro-t ér, valamilyen rögzített Euroárfolyamon!
- Tároljuk el ezt az értéket is!
- Írassuk ki hogy a megadott Forint összeg hány Euro-t ér szép, „x HUF = y EUR” formában, mindkét értéket megfelelő pontossággal!
- Alkalmazzuk a `main()` függvénnyel kapcsolatos „jó gyakorlat”-ot („*good practice*”)!

Pénzváltás

penzvalto.py (v1.0) [1–16]

```
1#!/usr/bin/env python3
2"""
3Programozás, Alapjai, előadás, példaprogram
4SZTE, TTIK, Informatikai Intézet, Szoftverfejlesztés, Tanszék
5Gergely, Tamás, gertom@inf.u-szeged.hu, 2024-05-31
6cc-by-nc-nd
7
8Pénzváltás, rögzített, árfolyamon
9"""
10def main():
11    forint = float(input("Hány forintot váltatsunk? "))
12    euro = forint / 396.717 # 2025-08-28
13    print(f"{forint:.0f} HUF = {euro:.2f} EUR")
14
15if __name__ == "__main__":
16    main()
```

Tartalomjegyzék

- 
- Bemutakozás**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)**
 - Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - Python sztringek**
 - Algoritmus vezérlése
 - A Python nyelv utasításai
 - Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - Python listák**
 - Python list és tuple adattípusok
 - Python listaszerű adattípusok műveletei
 - Basename és eltelt idő
 - Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - Python iterációk**
 - Csúszóátlag számítása
 - Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtávolabbi pontjai
 - Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - Vezérlési szerkezetek**
 - Eljárásvezérlés
 - Python fájlkezelés**
 - Fájlkezelés
 - Vezérlési szerkezetek**
 - Eljárásvezérlés
 - Python formátált IO**
 - Python formátált IO
 - Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - Python típusok**
 - Python numerikus típusok
 - Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - Python típusok**
 - További Python típusok
 - Python function adattípus
 - Absztrakt adattípusok**
 - Összetett adattípusok
 - Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

Pénzváltás

penzvalto.py (v2.0) [1–20]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-05-31
6cc-by-nc-nd
7
8Pénzváltás rögzített árfolyamon, 'konstans' és függvény segítségével
9"""
10HUFEUR = 396.717                                     # 2025-08-28
11
12def valtas(forint, arfolyam):
13    return forint / arfolyam
14
15def main():
16    forint = float(input("Hány forintot váltstunk? "))
17    print(f"{forint:.0f} HUF = {valtas(forint, HUFEUR):.2f} EUR")
18
19if __name__ == "__main__":
20    main()
```

- A probléma nem változott.
- A feladat: oldjuk meg „szebben”.



- A pénzváltó programunk funkcionálisan működjön úgy, ahogy eddig!



Pénzváltás

penzvalto.py (v1.0) [1–16]

```
1#!/usr/bin/env python3
2"""
3Programozás, Alapjai, előadás, példaprogram
4SZTE, TTIK, Informatikai Intézet, Szoftverfejlesztés, Tanszék
5Gergely, Tamás, gertom@inf.u-szeged.hu, 2024-05-31
6cc-by-nc-nd
7
8Pénzváltás, rögzített, árfolyamon
9"""
10def main():
11    forint = float(input("Hány forintot váltatsunk? "))
12    euro = forint / 396.717 # 2025-08-28
13    print(f"{forint:.0f} HUF = {euro:.2f} EUR")
14
15if __name__ == "__main__":
16    main()
```

- A pénzváltó programunk funkcionálisan működjön úgy, ahogy eddig!
- A HUF/EUR árfolyamot emeljük ki, lássuk el külön azonosítóval!



Pénzváltás

penzvalto.py (v1.1) [1–18]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-07-01
6cc-by-nc-nd
7
8Pénzváltás rögzített árfolyamon, 'konstans' és függvény segítségével
9"""
10HUFEUR = 396.717                                # 2025-08-28
11
12def main():
13    forint = float(input("Hány forintot váltunk? "))
14    euro = forint / HUFEUR
15    print(f"{forint:.0f} HUF = {euro:.2f} EUR")
16
17if __name__ == "__main__":
18    main()
```

A return utasítás

- Minden függvényben lehet egy vagy több return utasítás.
- Ha a függvényben egy ilyen utasítást hajtunk végre, akkor a függvény értékének kiszámítása befejeződik. A hívás helyén a függvényhívás kifejezés a return által kiszámított értéket veszi fel.
- Ha egy függvényben nincs return utasítás, vagy a függvény nem ennek végrehajtásával ér véget, az olyan, mintha egy return None-t hajtottunk volna végre.

```
def f(a, b):  
    return a + b
```

- A pénzváltó programunk funkcionálisan működjön úgy, ahogy eddig!
- A HUF/EUR árfolyamot emeljük ki, lássuk el külön azonosítóval!
- A váltás folyamatára készítsünk egy függvényt, és használjuk is!



Pénzváltás

penzvalto.py (v1.2) [1–21]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-07-01
6cc-by-nc-nd
7
8Pénzváltás rögzített árfolyamon, 'konstans' és függvény segítségével
9"""
10HUFEUR = 396.717                                # 2025-08-28
11
12def valtas(forint, arfolyam):
13    return forint / arfolyam
14
15def main():
16    forint = float(input("Hány forintot váltsunk? "))
17    euro = valtas(forint, HUFEUR)
18    print(f"{forint:.0f} HUF = {euro:.2f} EUR")
19
20if __name__ == "__main__":
21    main()
```

- A pénzváltó programunk funkcionálisan működjön úgy, ahogy eddig!
- A HUF/EUR árfolyamot emeljük ki, lássuk el külön azonosítóval!
- A váltás folyamatára készítsünk egy függvényt!
- Egyszerűsítsünk: hagyjuk el a felesleges változót!



Pénzváltás

penzvalto.py (v2.0) [1–20]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-05-31
6cc-by-nc-nd
7
8Pénzváltás rögzített árfolyamon, 'konstans' és függvény segítségével
9"""
10HUFEUR = 396.717                                     # 2025-08-28
11
12def valtas(forint, arfolyam):
13    return forint / arfolyam
14
15def main():
16    forint = float(input("Hány forintot váltstunk? "))
17    print(f"{forint:.0f} HUF = {valtas(forint, HUFEUR):.2f} EUR")
18
19if __name__ == "__main__":
20    main()
```